

ADVANCED OOP WITH JAVA

Instructor:



◇ **Polymorphism**

- ✓ Types of Polymorphism
- ✓ Method Overloading
- ✓ Method Overriding
- ✓ Static and Dynamic binding

◇ **Abstraction**

- ✓ Abstract class
- ✓ Interfaces



Section 1

POLYMORPHISM

4 major principles of OOP

- ❖ **Inheritance**
- ❖ **Encapsulation**
- ❖ **Polymorphism**
- ❖ **Abstraction**

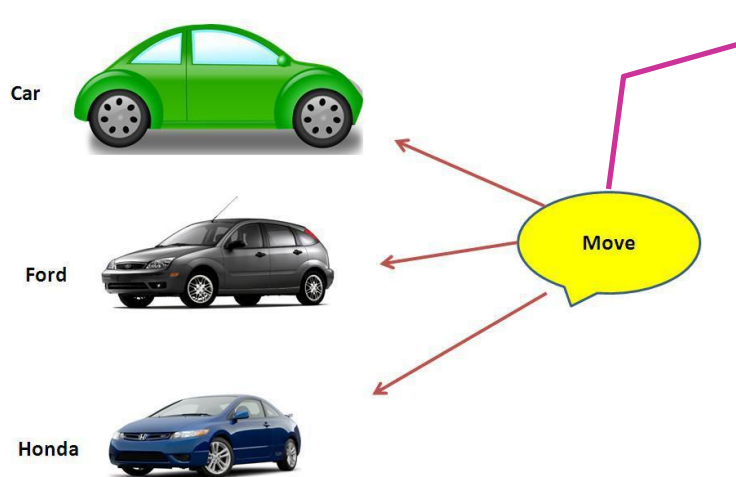
- In object-oriented programming, **polymorphism** (from the Greek (Hy Lạp) meaning "**having multiple forms**").
 - ✓ Polymorphism is derived from 2 greek words: **poly** and **morphs**.
 - ✓ The word "**poly**" means *many* and "**morphs**" means *forms*.



- There are two types of polymorphism in java:
 - ✓ **compile time polymorphism**: method **overloading**.
 - ✓ **runtime polymorphism**: method **overriding**.



one name, many forms.



Polymorphism is a generic term for **having many forms**.

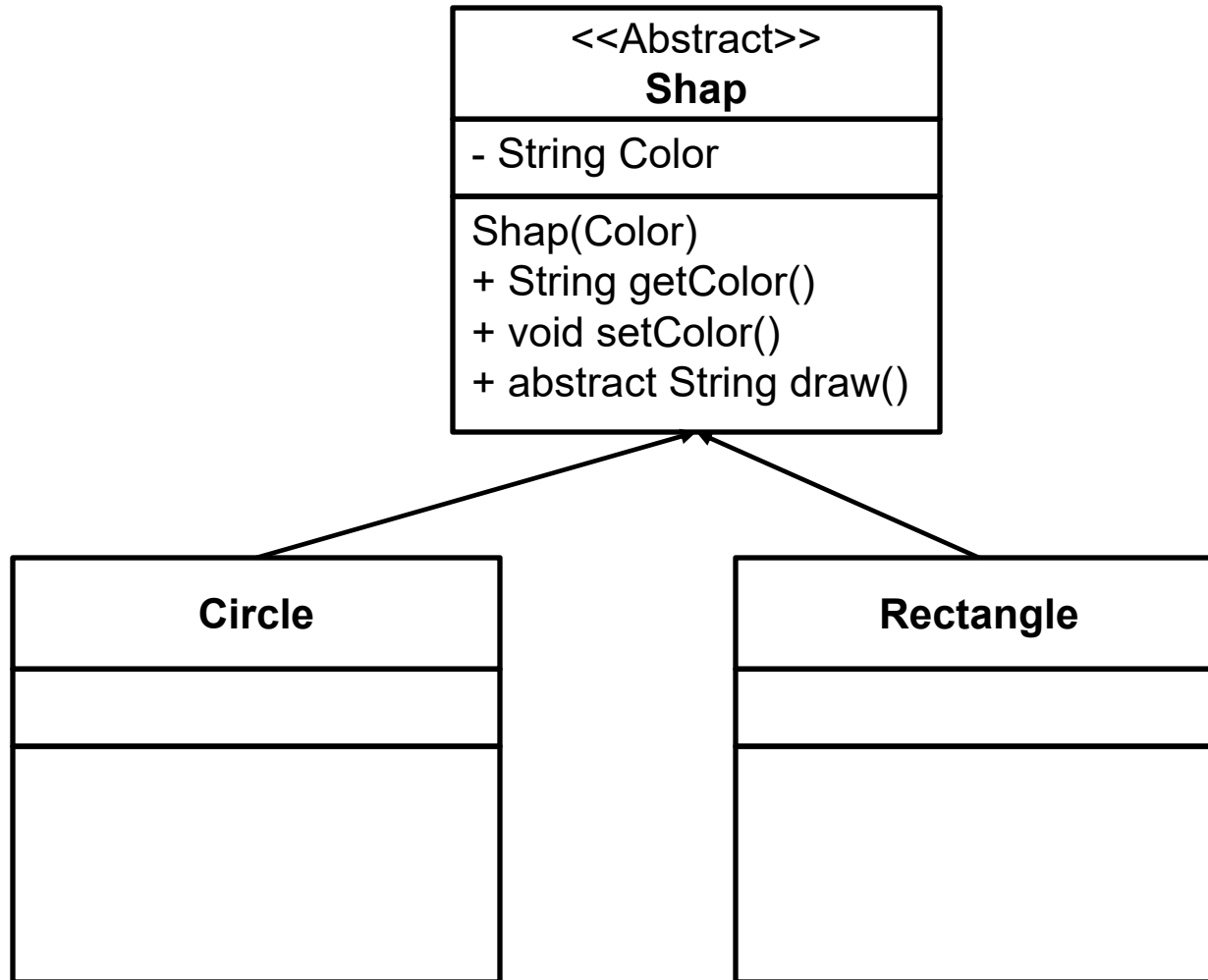
- Car uses normal engine to move
- Ford uses V engine to move
- Honda uses i-vtec technology to move

■ You can use the **same name** for **several different things**

✓ the compiler **automatically figures out** which **version** you wanted.

■ There are several forms of polymorphism supported in Java, **shadowing**, **overriding**, and **overloading**.

Polymorphism example



Polymorphism example (1/4)

```
public abstract class Shape {  
    private String color;  
  
    public Shape(String color) {  
        this.setColor(color);  
    }  
  
    public String getColor() {  
        return Color;  
    }  
  
    public void setColor(String color) {  
        Color = color;  
    }  
    // abstract method  
    abstract public String draw();  
}
```

Polymorphism example (2/4)

```
public class Circle extends Shape {  
    /**  
     * @param color  
     */  
    public Circle(String color) {  
        super(color);  
    }  
  
    @Override  
    public String draw() {  
        return "I'm a " + this.getColor() + " circle.";  
    }  
}
```

Polymorphism example (3/4)

```
public class Rectangle extends Shape {  
    /**  
     * @param color  
     */  
    public Rectangle(String color) {  
        super(color);  
    }  
  
    @Override  
    public String draw() {  
        return "I'm a " + this.getColor() + " rectangle.";  
    }  
}
```

Polymorphism example (4/4)

```
public class PolymorphismExample {  
    private List<Shape> shapes = new ArrayList<Shape>();  
    public PolymorphismExample() {  
        Shape myFirstCircle = new Circle("Red");  
        Circle mySecondCircle = new Circle("Blue");  
        Rectangle myFirstRectangle = new Rectangle("Green");  
        shapes.add(myFirstCircle);  
        shapes.add(mySecondCircle);  
        shapes.add(myFirstRectangle);  
    }  
    public List<Shape> getShapes() {  
        return shapes;  
    }  
    public static void main(String[] args) {  
        PolymorphismExample example = new PolymorphismExample();  
        for (Shape shape : example.getShapes()) {  
            System.out.println(shape.draw());  
        }  
    }  
}
```

Output:

```
I'm a Red circle.  
I'm a Blue circle.  
I'm a Green rectangle.
```

- ❖ There are two types of polymorphism in java:
 - ✓ **Static Polymorphism** also known as **compile time** polymorphism
 - ✓ **Dynamic Polymorphism** also known as **runtime** polymorphism
- ❖ **Compile time Polymorphism** (or Static polymorphism)
 - ✓ Polymorphism that is resolved during compiler time is known as static polymorphism. Method overloading is an example of compile time polymorphism.
 - ✓ **Method Overloading**: This allows us to have **more than one method** having the **same name**, if the parameters of methods are different in number, sequence and data types of parameters.

- ❖ Overloaded methods let you reuse the **same method name** in a class, but with **different arguments** (and optionally, a different return type).
- ❖ There are two ways to overload the method in java:
 - ✓ By changing number of arguments
 - ✓ By changing the data type
- ❖ In a subclass,
 - ✓ you can **overload** the methods inherited **from the superclass**.
 - ✓ such overloaded methods **neither hide nor override** the superclass methods
 - ✓ they are **new methods**, **unique** to the subclass.

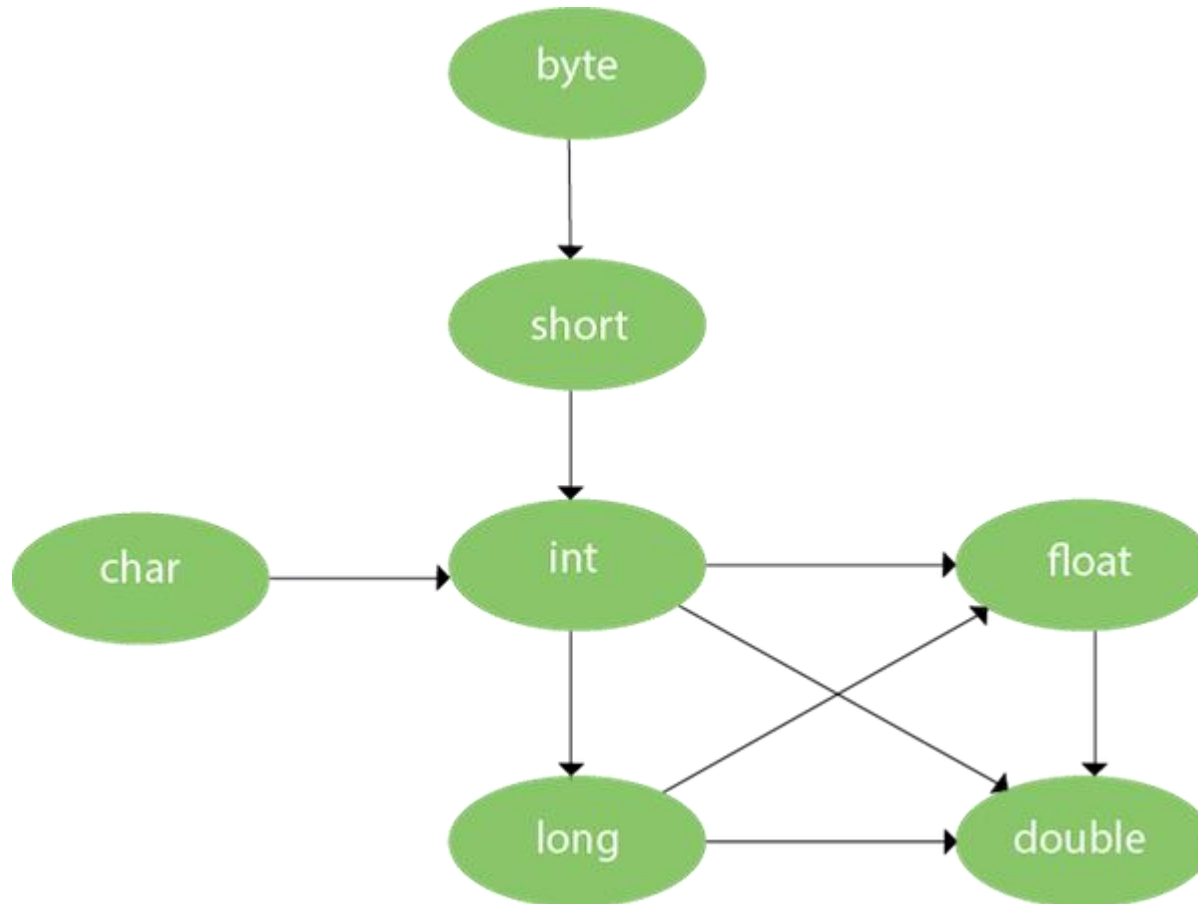
- ❖ Some main rules for overloading a method:
 - ✓ Overloaded methods **must change the argument list**.
 - ✓ Overloaded methods **can change the return type**.
 - ✓ Overloaded methods **can change the access modifier**.
 - With override: Cannot reduce the visibility
 - ✓ A method can be overloaded in the **same class** or in a **subclass**.

NOTE: WHEN I SAY METHOD SIGNATURE I AM NOT TALKING ABOUT RETURN TYPE OF THE METHOD, FOR EXAMPLE IF TWO METHODS HAVE SAME NAME, SAME PARAMETERS AND HAVE DIFFERENT RETURN TYPE, THEN THIS IS NOT A VALID METHOD OVERLOADING EXAMPLE. THIS WILL THROW COMPILATION ERROR.

Types of Polymorphism

```
public class SimpleCalculator {  
    int add(int number1, int number2) {  
        return number1 + number2;  
    }  
  
    int add(int number1, int number2, int number3) {  
        return number1 + number2 + number3;  
    }  
  
    double add(double number1, double number2) {  
        return number1 + number2;  
    }  
}  
  
public class TestSimpleCalculator {  
    public static void main(String[] args) {  
        SimpleCalculator simpleCalculator = new SimpleCalculator();  
  
        System.out.println(simpleCalculator.add(10, 20));  
        System.out.println(simpleCalculator.add(10, 20, 30));  
        System.out.println(simpleCalculator.add(12.5, 20.5));  
    }  
}
```


- ❖ One type is promoted to another implicitly if no matching datatype is found:



- A program calculates and displays bonus amounts to pay various types of employees. There are 3 separate departments, numbered 1, 2, and 3.
 - ✓ **Department 1** employees are paid a bonus based on their sales: If their sales amount is over \$5000 they get 5% of those sales, otherwise they get nothing.
 - ✓ **Department 2** employees are paid a bonus based on the number of units they sell: They get \$100 per unit sold, and an extra \$50 per unit if they sell more than 25 units; if they sell no units, they get nothing.
 - ✓ **Department 3** employees assemble parts in the plant and are paid a bonus of 10 cents per part if they reach a certain level: Part-time employees must assemble more than 250 parts to get the 10-cent-per-part bonus, and full-time employees must assemble more than 700.
- Write a set of 3 overloaded methods called `getBonus()` that works with the program below, according to the specifications described above.



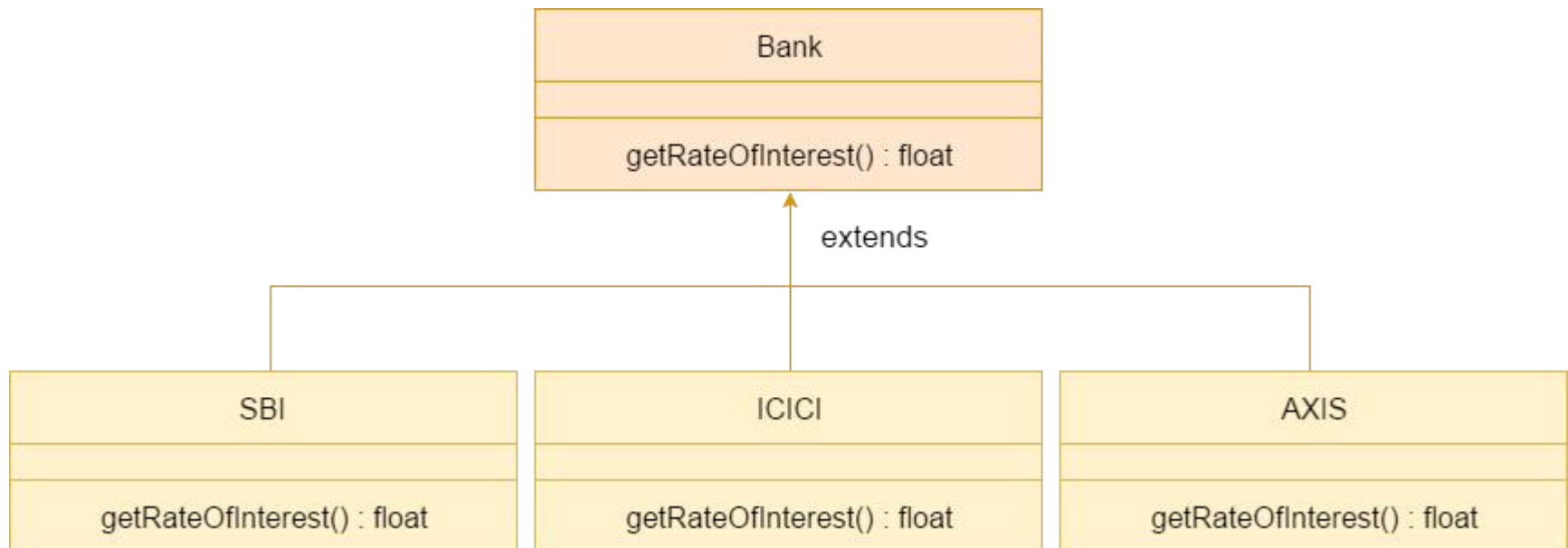
- ❖ Declaring a method in **subclass** which is already present in **parent class** is known as method overriding.
- ❖ The main advantage of method overriding is:
 - ✓ the class can give its own specific implementation to a inherited method without even modifying the parent class(base class).
- ❖ **Rules of method overriding in Java:**
 - ✓ **Method name:** method must have the same name as in the parent class.
 - ✓ **Argument list:** must be same as that of the method in parent class,
 - ✓ **Access Modifier:** cannot be more restrictive than the overridden method of parent class.

Overriding Examples

```
class Mammal {  
    String makeNoise() {  
        return "generic noise";  
    }  
}  
  
class Zebra extends Mammal {  
    String makeNoise() {  
        return "bray";  
    }  
}  
  
public class ZooKeeper {  
    public static void main(String[] args) {  
        Mammal m = new Zebra();  
        System.out.println(m.makeNoise());  
    }  
}
```

A real example of Method Overriding

- ❖ Consider a scenario where **Bank** is a class that provides functionality to get the rate of interest. However, the rate of interest varies according to banks.
- ❖ For example, **SBI**, **ICICI** and **AXIS** banks could provide **8%**, **7%**, and **9%** rate of interest.



- ❖ This is called **shadowing**—**name** in class **Dog** shadows **name** in class **Animal**

```
public class Animal {
    String name = "Animal";
    public void speak() {
        System.out.println("generic speak!");
    }
    public static void main(String args[]) {
        Animal animal1 = new Animal();
        Animal animal2 = new Dog();
        Dog dog = new Dog();
        System.out.println(animal1.name + " " + animal2.name + " " + dog.name);
    }
}
class Dog extends Animal {
    String name = "Dog";
    public void speak() {
        System.out.println("dog speak!");
    }
}
```

Animal Animal Dog

❖ Static binding:

- ✓ The binding which can be resolved at compile time by compiler is known as static or early binding.
- ✓ The binding of static, private and final methods is compile-time. The reason is that these methods cannot be overridden and the type of the class is determined at the compile time.

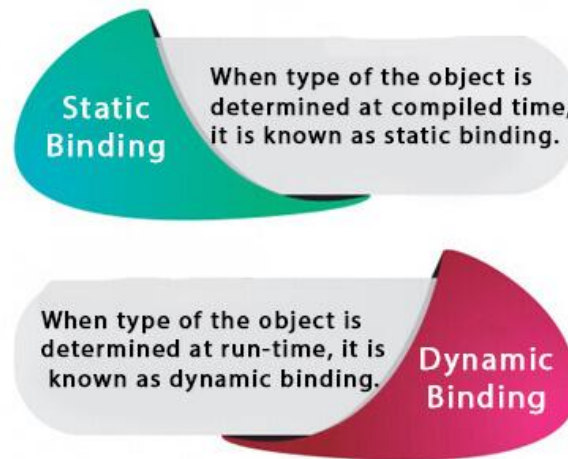
❖ Dynamic binding:

- ✓ When compiler is not able to resolve the call/binding at compile time, such binding is known as Dynamic or late Binding.
- ✓ Method Overriding is a perfect example of dynamic binding as in overriding both parent and child classes have same method and in this case the **type of the object** determines which method is to be executed.
- ✓ The type of object is determined at the run time so this is known as dynamic binding.

Static and Dynamic binding

Static binding	Dynamic binding
Happens at Compile time	Happens at Runtime
Actual object is not used	Actual object is used
Also known as Early binding	Also known as Late binding
Speed is high	Speed is low
Ex: - Method Overloading	Ex: - Method Overriding

Static vs Dynamic Binding



- ❖ If a subclass defines a **class method** with the **same signature** as a **class method** in the superclass, the method in the subclass hides the one in the superclass.

```
3 public class Shape {  
4     public static void testClassMethod() {  
5         System.out.println("The class method in Shape.");  
6     }  
7  
8     public void testInstanceMethod() {  
9         System.out.println("The instance method in Shape.");  
10    }  
11  
12 }
```

```
3 public class Circle extends Shape {  
4     public static void testClassMethod() {  
5         System.out.println("The class method in Circle.");  
6     }  
7  
8     public void testInstanceMethod() {  
9         System.out.println("The instance method in Circle.");  
10    }  
11 }
```

Static binding (Hiding method)

```
public class TestOverridingAndHiding {  
    public static void main(String[] args) {  
        Circle myCircle = new Circle();  
        Shape myShape = myCircle;  
  
        Shape.testClassMethod();  
        myShape.testInstanceMethod();  
    }  
}
```

Output:

The class method in Shape.

The instance method in Circle.

```
public class Boy extends Human {
    public static void walk() {
        System.out.println("Boy walks");
    }

    public static void main(String args[]) {

        /* Reference is of Human type and object is
         * Boy type
         */
        Human obj = new Boy();
        /* Reference is of HUmAn type and object is
         * of Human type.
         */
        Human obj2 = new Human();
        obj.walk();
        obj2.walk();
    }
}

class Human {
    public static void walk() {
        System.out.println("Human walks");
    }
}
```

OUTPUT:

```
Human walks
Human walks
```

```
public class Boy extends Human {
    public void walk() {
        System.out.println("Boy walks");
    }

    public static void main(String args[]) {

        /* Reference is of Human type and object is
        * Boy type
        */
        Human obj = new Boy();
        /* Reference is of Human type and object is
        * of Human type.
        */
        Human obj2 = new Human();
        obj.walk();
        obj2.walk();
    }
}

class Human {
    public void walk() {
        System.out.println("Human walks");
    }
}
```

OUTPUT:

```
Boy walks
Human walks
```

- ❖ The **covariant return type** specifies that the return type may vary in the same direction as the subclass.
- ❖ *Before Java5, it was not possible to override any method by changing the return type.*
- ❖ But now, since Java5, it is possible to override method by changing the return type if subclass overrides any method whose return type is **Non-Primitive** but it changes its return type to **subclass type**.
- ❖ **Example:**

```
class A{
    A get(){return this;}
}

class B extends A{
    B get(a){return this;}
    void message(){System.out.println("welcome to covariant return type");
}

    public static void main(String args[]){
        new B().get().message();
    }
}
```

- ❖ The **java instanceof operator** is used to test whether the object is an instance of the specified type (class or subclass or interface).
- ❖ The **instanceof** is also known as type *comparison operator* because it compares the instance with type.
 - ✓ It returns either **true** or **false**.
 - ✓ If we apply the instanceof operator with any variable that has null value, it returns false.
- ❖ **Example:**

```
class Animal{}  
class Dog extends Animal{//Dog inherits Animal  
    public static void main(String args[]){  
        Animal a = new Animal();  
        Dog d = new Dog();  
        System.out.println(a instanceof Animal);//true  
        System.out.println(d instanceof Animal);//?  
    }  
}
```

Output:

```
true  
true
```

- ❖ When Subclass type refers to the object of Parent class, it is known as downcasting.
- ❖ Let's see the example, downcasting be performed without the use of instanceof operator:
 - ✓ The actual object that is referred by a, is an object of Dog class. **So if we downcast it, it is fine.**

```
class Animal {  
}  
  
public class Dog extends Animal {  
  
    static void cast(Animal a) {  
        Dog d = (Dog) a;  
        System.out.println("Downcasting performed");  
    }  
  
    public static void main(String[] args) {  
        Animal animal = new Dog();  
        cast(animal);  
    }  
}
```

❖ But what will happen if we write:

```
Animal animal = new Animal();  
Dog d = cast(animal);
```

❖ You faced with an exception:

Exception in thread "main" [java.lang.ClassCastException: fa.training.jpe.Animal cannot be cast to fa.training.jpe.Dog](#)

❖ To resolve this problem, should perform downcasting with java **instanceof** operator. Re-write code:

```
class Animal {}  
  
public class Dog extends Animal {  
  
    static void cast(Animal a) {  
        if (a instanceof Dog) {  
            Dog d = (Dog) a;  
            System.out.println("Downcasting performed");  
        }  
    }  
  
    public static void main(String[] args) {  
        Animal animal = new Animal();  
        cast(animal);  
    }  
}
```

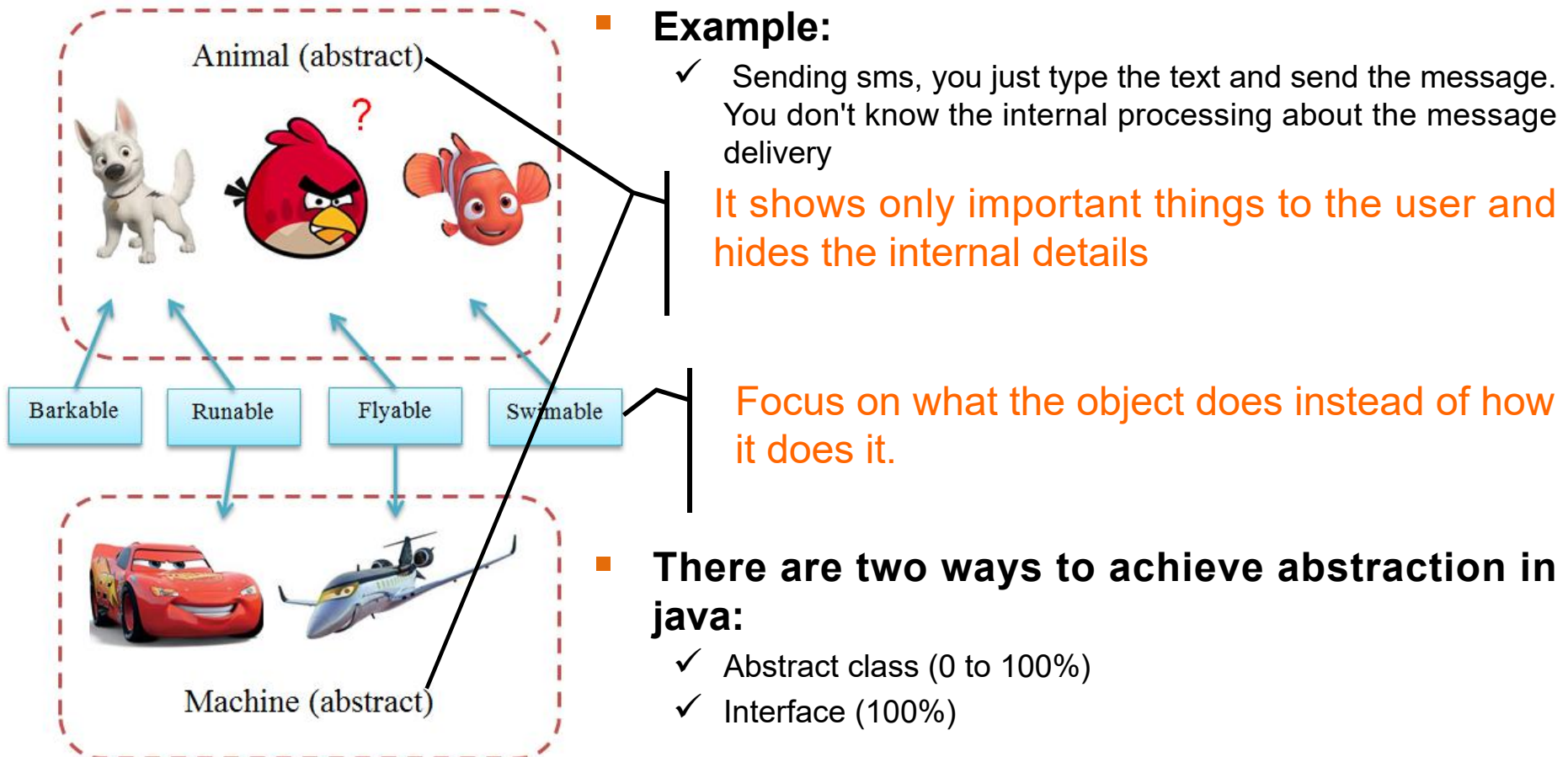

Section 2

ABSTRACTION

4 major principles of OOP

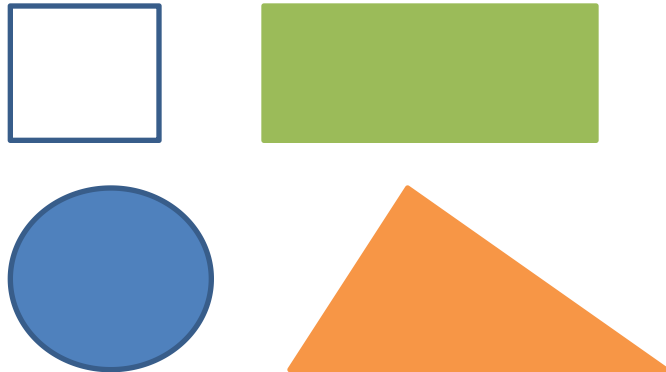
- ❖ **Inheritance**
- ❖ **Encapsulation**
- ❖ **Polymorphism**
- ❖ **Abstraction**

- ❖ **Abstraction** is a process of hiding the implementation details and showing only functionality to the user.

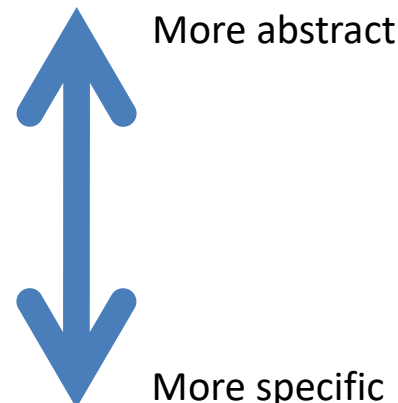
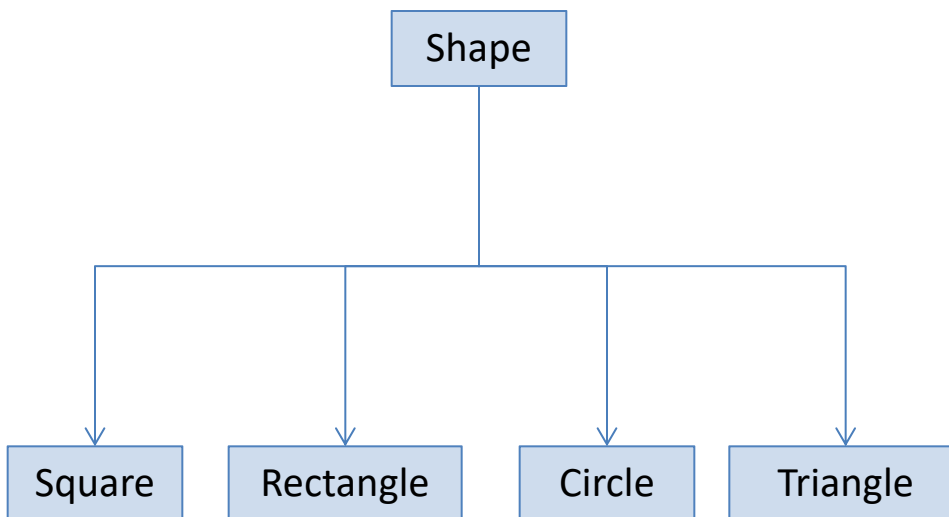


- ❖ Abstraction is used to **manage complexity**.
 - ✓ For example: *"sending SMS where you type the text and send the message, you don't know the internal processing about the message delivery"*.
- ❖ Abstraction lets you focus on what the object does instead of how it does it.
- ❖ There are two ways to achieve abstraction in java
 - ✓ Abstract class (0 to 100%)
 - ✓ Interface (100%)

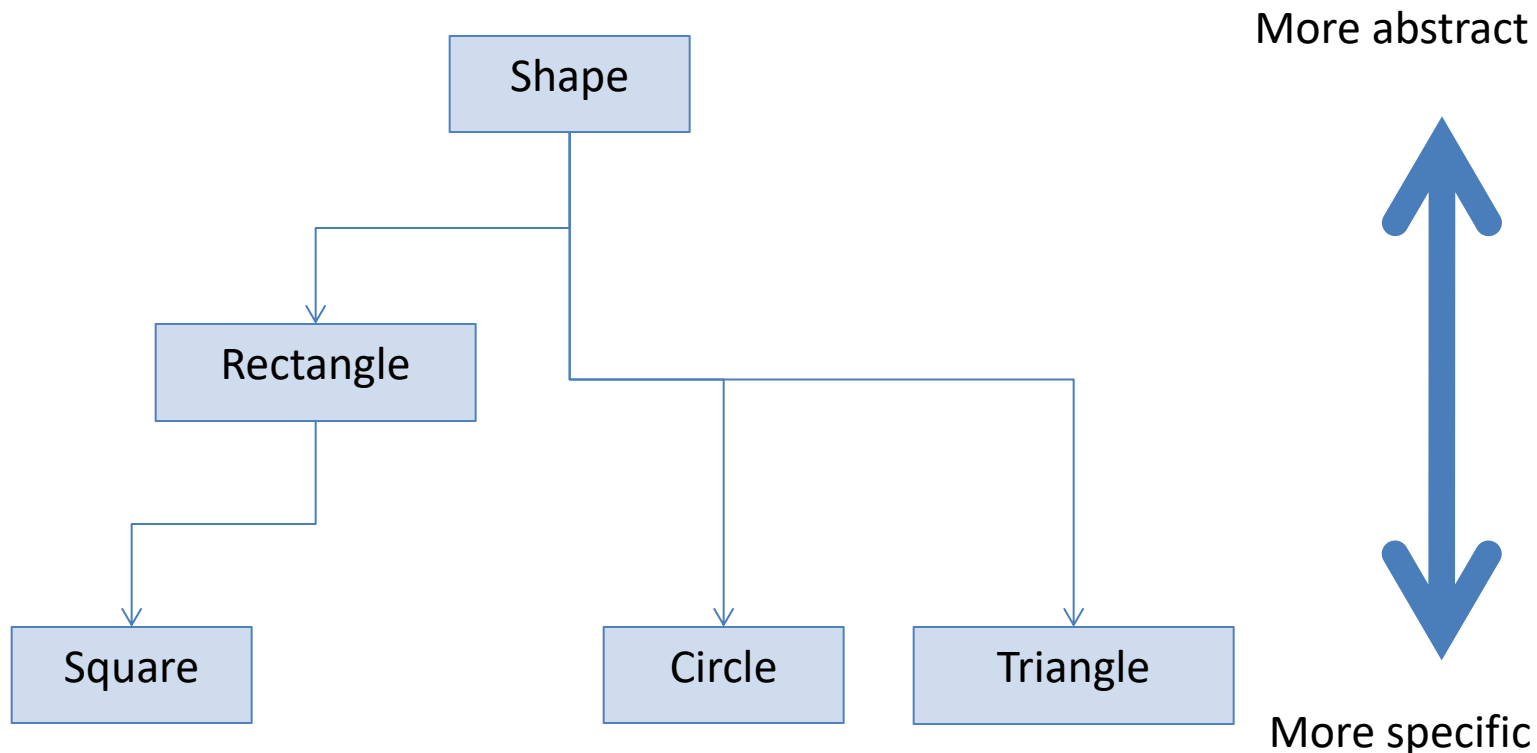
❖ Abstraction is used to **manage complexity**.



- All **Square, Rectangle, Circle and Triangle**:
 - ✓ Have *color*
 - ✓ Can *display*
- A shape may has all above characteristics:
 - ✓ So we call “**Shape**” is an abstract type of Square, Rectangle, Circle and Triangle.



- ❖ Then, we analysis deeper:
 - ✓ A **rectangle** has four sides with lengths **w** and **h**
 - ✓ A **square** has all of the characteristics of a rectangle; in addition, **w = h**
- ❖ So, **square** is a type of rectangle, or, rectangle can be an abstract type of square



- ❖ A class which is declared as abstract is known as an **abstract class**.
 - ✓ It can have abstract and non-abstract methods.
 - ✓ It needs to be extended and its method implemented.
 - ✓ It cannot be instantiated.
- ❖ **Rules for Java Abstract class:**

- 1 • An abstract class must be declared with an abstract keyword.
- 2 • It can have abstract and non-abstract methods.
- 3 • It **cannot** be instantiated.
- 4 • It can have constructors and static methods also.
- 5 • It can have final methods which will force the subclass not to change the body of the method.

- ❖ An abstract class can not be **instantiated** (you are not allowed to create **object** of Abstract class), but they can be subclassed.

```
Shape shape = new Circle();
```

- ❖ When an abstract class is subclassed, the subclass usually provides implementations for all of the abstract methods in its parent class.

✓ **However**, if it does not, the subclass must also be declared abstract.

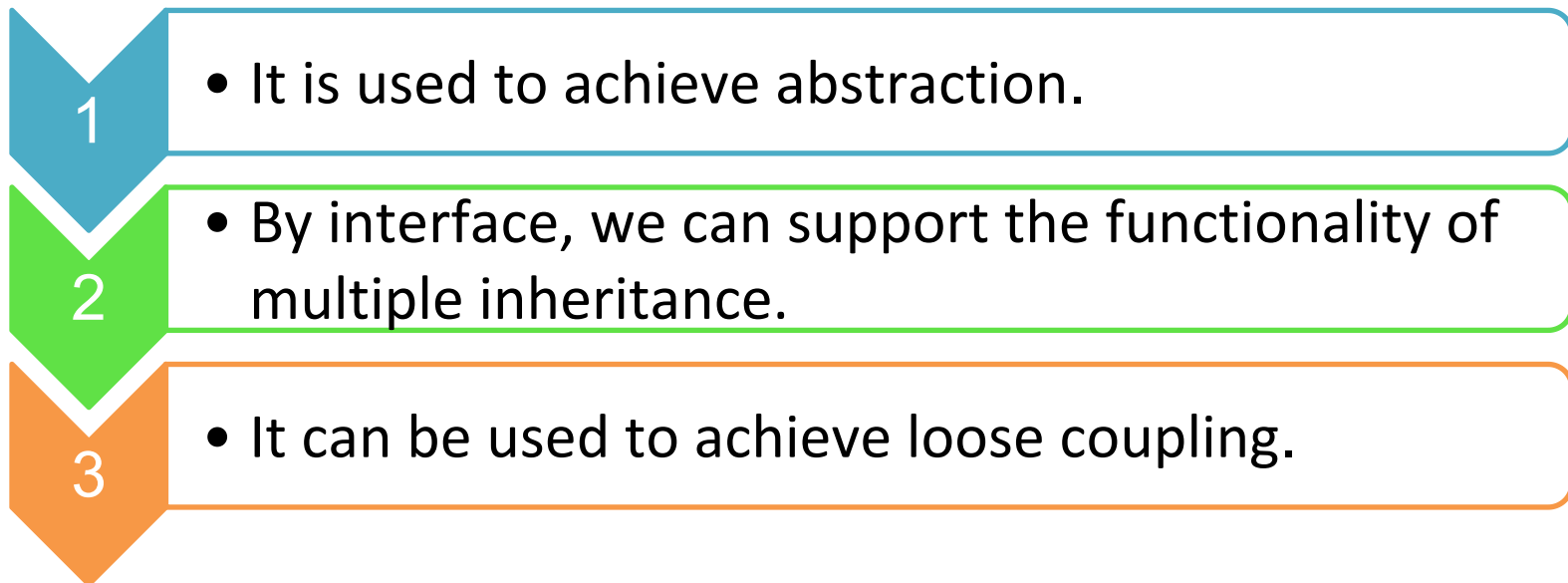
- ❖ **Abstract method:**

- ✓ An abstract method is a method that is declared **without an implementation**.

Example: `abstract void moveTo(int x, int y);`

- ✓ If a class includes abstract methods, the class itself **must be declared abstract**.
- ✓ All of the methods in an interface are implicitly abstract: so the abstract modifier is **not used with interface methods**.

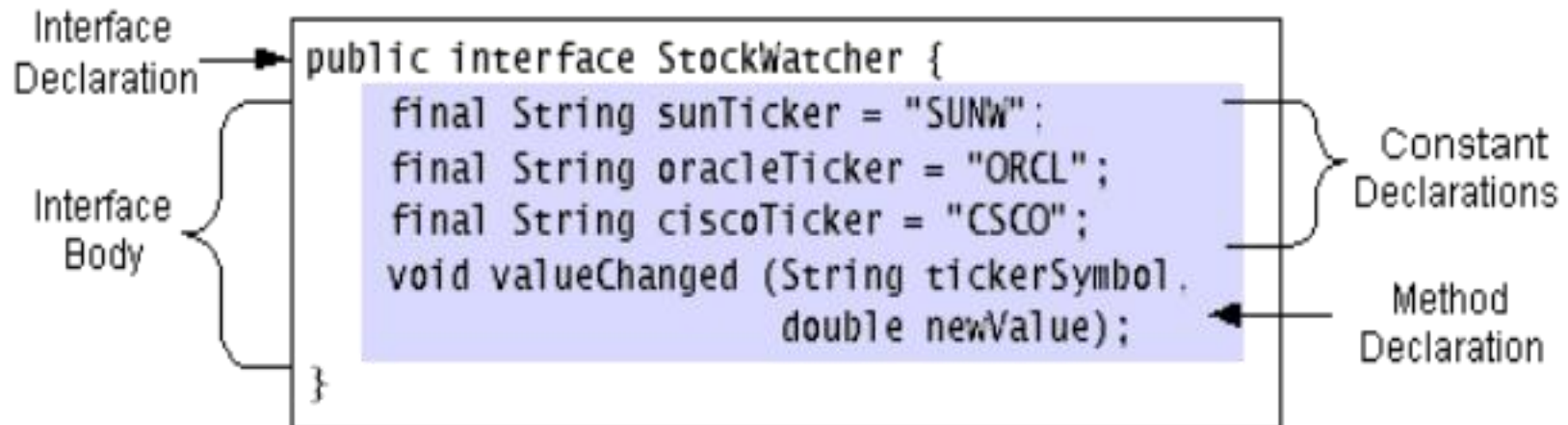
- ❖ An **interface in Java** is a blueprint of a class. It has static constants and abstract methods.
- ❖ The interface in Java is *a mechanism to achieve [abstraction](#)*.
 - ✓ There can be only abstract methods in the Java interface, not method body.
 - ✓ It is used to achieve abstraction and multiple [inheritance in Java](#).
- ❖ **Why use Java interface?**



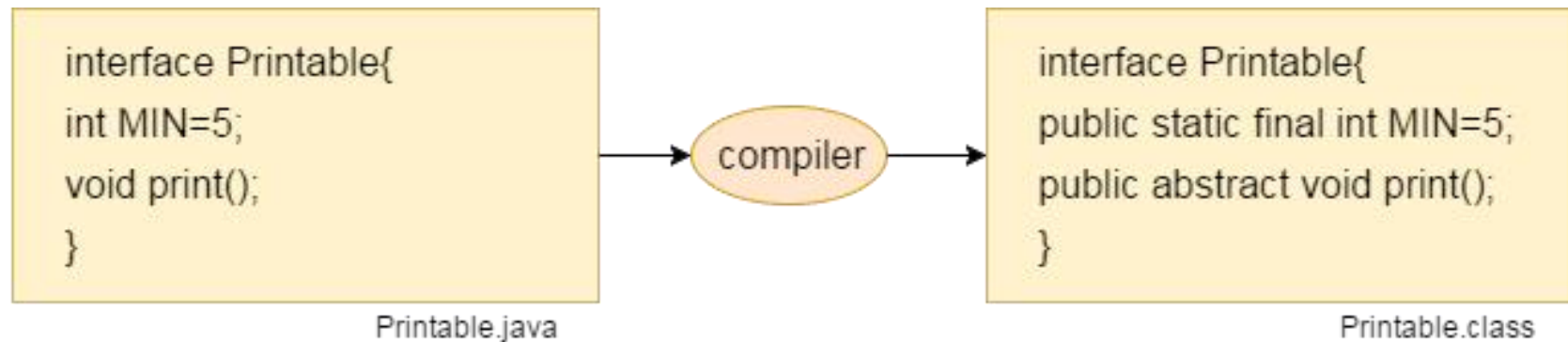
❖ Syntax:

```
[public] interface <InterfaceName>[extends SuperInterface] {  
    // Interface Body  
}
```

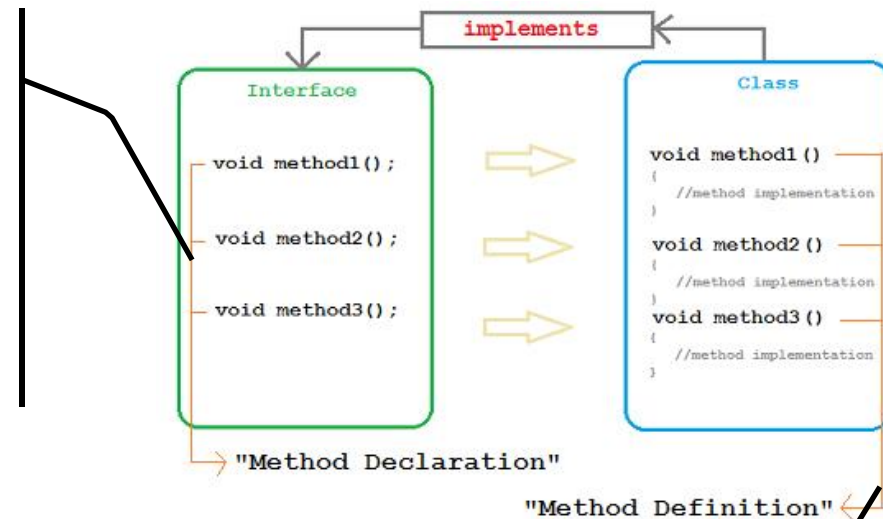
❖ Example:



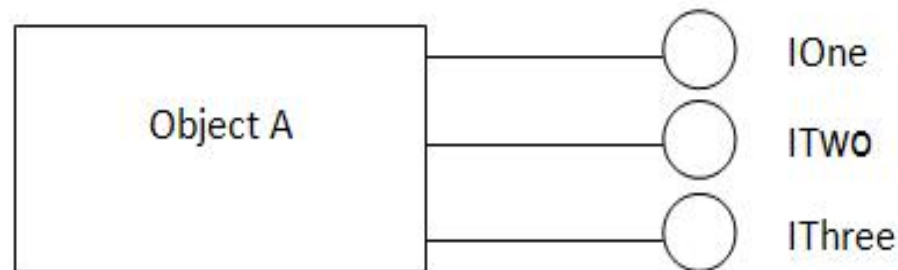
- ❖ The Java compiler adds **public** and **abstract** keywords before the interface method.
- ❖ Moreover, it adds **public**, **static** and **final** keywords before data members.
- ❖ Consider the following figure:



- ✓ An interface is a definition of method **prototypes** and possibly **some constants** (static final fields).
- ✓ An interface **does not** include the **implementation** of any methods.



- ✓ A class can **implement** an interface, this means that it provides implementations for all the methods in the interface.
- ❖ Java classes can implement any number of interfaces (multiple interface inheritance).



Java Interface Example: Bank

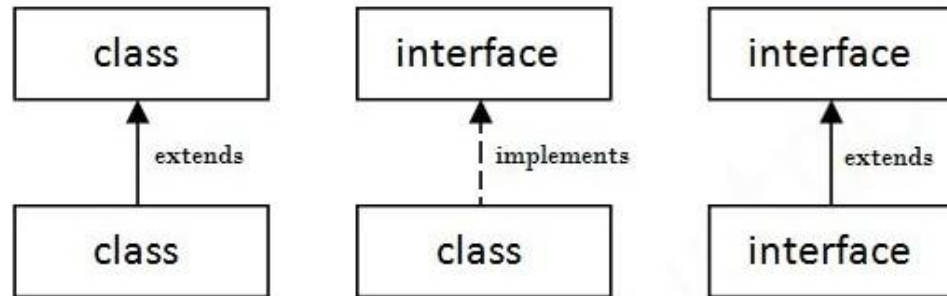
- ❖ Let's see another example of java interface which provides the implementation of Bank interface.

```
interface Bank {  
    float rateOfInterest();  
}  
  
class SBI implements Bank {  
    public float rateOfInterest() {  
        return 9.15f;  
    }  
}  
  
class PNB implements Bank {  
    public float rateOfInterest() {  
        return 9.7f;  
    }  
}  
  
public class TestInterface {  
  
    public static void main(String[] args) {  
  
        Bank b = new SBI();  
        System.out.println("ROI: " + b.rateOfInterest());  
    }  
}
```

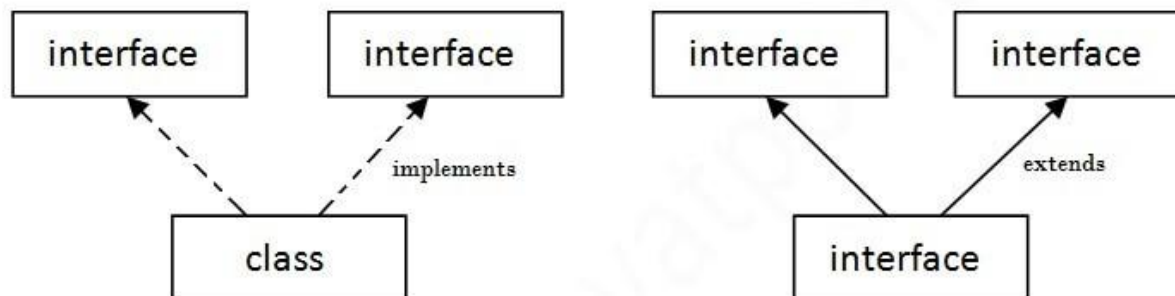
Output:

ROI: 9.15

- ❖ As shown in the figure given below, a class extends another class, an interface extends another interface, but a **class implements an interface**.



- ❖ Multiple inheritance in Java by interface



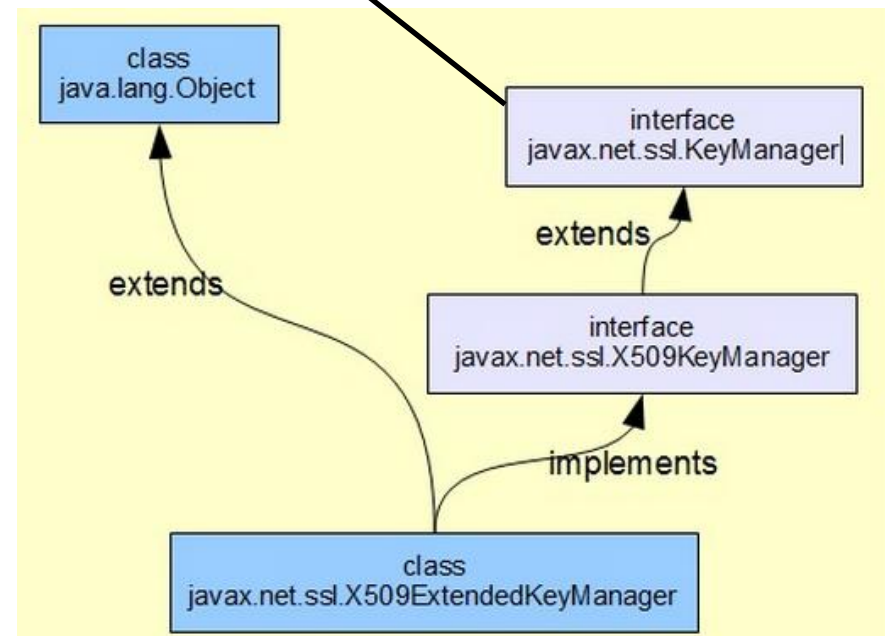
Multiple Inheritance in Java

Interfaces can be extended

❖ Example:

```
public interface Forward {  
    void drive();  
}  
  
public interface Stop {  
    void park();  
}  
  
public interface Speed {  
    void turbo();  
}  
  
public class GearBox {  
    public void move() {  
    }  
}
```

one interface can extend another.



❖ Example:

class Automatic **extends** GearBox **implements** Forward, Stop, Speed

```
{  
    public void drive() {  
        System.out.println("drive()");  
    }  
    public void park() {  
        System.out.println("park()");  
    }  
    public void turbo() {  
        System.out.println("turbo()");  
    }  
    public void move() {  
        System.out.println("move()");  
    }  
}
```


❖ Example:

```
public class Car {  
    public static void cruise(Forward x) {  
        x.drive();  
    }  
    public static void park(Stop x) {  
        x.park();  
    }  
    public static void race(Speed x) {  
        x.turbo();  
    }  
    public static void move(GearBox x) {  
        x.move();  
    }  
    public static void main(String[] args) {  
        Automatic auto = new Automatic();  
        cruise(auto); // Interface Forward  
        park(auto); // Interface Stop  
        race(auto); // Interface Speed  
        move(auto); // class GearBox  
    }  
}
```

Multiple inheritance in Java by interface

- ❖ Multiple inheritance is not supported in the case of class because of ambiguity.
- ❖ However, it is supported in case of an interface because there is no ambiguity.
- ❖ It is because its implementation is provided by the implementation class.
- ❖ **Example:**

```
interface Printable {  
    void print();  
}  
  
interface Showable {  
    void print();  
}  
  
class A4 implements Printable, Showable {  
    public void print() {  
        System.out.println("Printing and showing document");  
    }  
}  
  
public class TestInterface2 {  
    public static void main(String[] args) {  
        A4 a4 = new A4();  
        a4.print();  
    }  
}
```

Java 8 Static Method in Interface

- ❖ Since Java 8, we can have method body in interface. But we need to make it **default method**.
- ❖ **Example:**

```
interface Drawable {  
    void draw();  
  
    default void msg() {  
        System.out.println("default method");  
    }  
}  
  
class Rectangle implements Drawable {  
    public void draw() {  
        System.out.println("drawing rectangle");  
    }  
}  
  
public class TestInterfaceDefault {  
    public static void main(String[] args) {  
        Drawable d = new Rectangle();  
        d.draw();  
        d.msg();  
    }  
}
```

Output:

```
drawing rectangle  
default method
```

Java 8 Static Method in Interface

- ❖ Since Java 8, we can have static method in interface.
- ❖ **Example:**

```
interface Drawable {  
    void draw();  
  
    static int cube(int x) {  
        return x * x * x;  
    }  
}  
  
class Rectangle implements Drawable {  
    public void draw() {  
        System.out.println("drawing rectangle");  
    }  
}  
  
public class TestInterfaceStatic {  
    public static void main(String[] args) {  
        Drawable d = new Rectangle();  
        d.draw();  
        System.out.println(Drawable.cube(3));  
    }  
}
```

Output:

```
drawing rectangle  
27
```



- ❖ **Abstract class** and **interface** both are used to achieve abstraction where we can declare the abstract methods. Abstract class and interface both can't be instantiated.
- ❖ But there are many differences between abstract class and interface that are given below.

No.	ABSTRACT CLASS	INTERFACE
1	ABSTRACT CLASS CAN HAVE ABSTRACT AND NON-ABSTRACT METHODS.	INTERFACE CAN HAVE ONLY ABSTRACT METHODS.
2	ABSTRACT CLASS DOESN'T SUPPORT MULTIPLE INHERITANCE.	INTERFACE SUPPORTS MULTIPLE INHERITANCE.
3	ABSTRACT CLASS CAN HAVE FINAL, NON-FINAL, STATIC AND NON-STATIC VARIABLES.	INTERFACE HAS ONLY STATIC AND FINAL VARIABLES.
4	ABSTRACT CLASS CAN HAVE STATIC METHODS, MAIN METHOD AND CONSTRUCTOR.	INTERFACE CAN'T HAVE STATIC METHODS, MAIN METHOD OR CONSTRUCTOR.
5	ABSTRACT CLASS CAN PROVIDE THE IMPLEMENTATION OF INTERFACE.	INTERFACE CAN'T PROVIDE THE IMPLEMENTATION OF ABSTRACT CLASS.
6	THE ABSTRACT KEYWORD IS USED TO DECLARE ABSTRACT CLASS.	THE INTERFACE KEYWORD IS USED TO DECLARE INTERFACE.
7	EXAMPLE: PUBLIC ABSTRACT CLASS SHAPE PUBLIC ABSTRACT VOID DRAW ;	EXAMPLE: PUBLIC INTERFACE DRAWABLE VOID DRAW ;

- Polymorphism, which means "**many forms**,"
 - ✓ is the *ability to treat an object of any subclass* of a base class as if it were an object of the base class.
- **Abstract class** is a class that **may contain abstract methods** and **implemented methods**.
 - ✓ An *abstract method* is one *without a body* that is declared with the reserved word `abstract`
- An **interface** is a collection of **constants** and **method declarations**.
 - ✓ When a class implements an interface, it must declare and provide a method body for each method in the interface

Thank you

